香 港 大 學
THE UNIVERSITY OF HONG KONG

应用场景与实践（UAV检测）

Part2:HTD-Mamba模型实战

Dr. Bing Huang
20/07/2026

香 港 大 學
THE UNIVERSITY OF HONG KONG

2026

目录

CONTENTS

01 | 数据介绍与预处理

02 | 模型介绍与代码讲解

03 | 实践与模型优化

1.1 环境配置

连接好AutoDL的V100卡后，建立虚拟环境，（htd-mamba名字可以自己取）

```
conda create -n htd-mamba python=3.10 -y

conda init bash
source ~/.bashrc

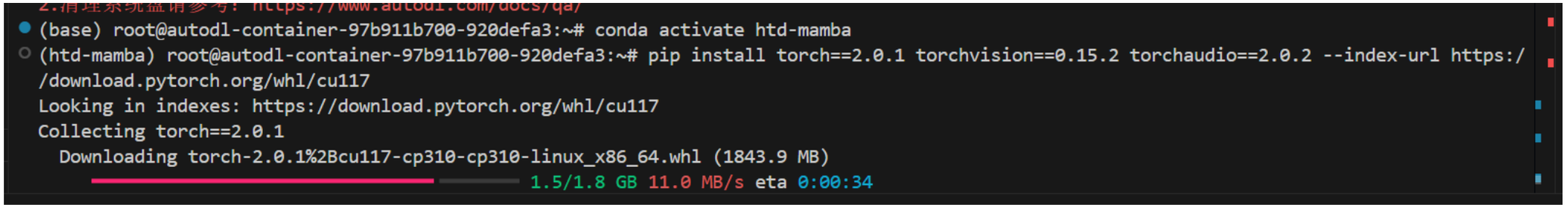
conda activate htd-mamba
```

1.1 环境配置

香 港 大 學
THE UNIVERSITY OF HONG KONG

装 PyTorch 2.0.1 + cu117

pip install torch==2.0.1 torchvision==0.15.2 torchaudio==2.0.2 --index-url https://download.pytorch.org/whl/cu117



1.1 环境配置

香 港 大 學
THE UNIVERSITY OF HONG KONG

安装相关依赖文件

cd HTD-muban # 换成你实际路径

pip install -r requirements.txt

```
(htd-mamba) root@autodl-container-97b911b700-920defa3:~/HTD-muban# pip install -r requirements.txt
Requirement already satisfied: lit in /root/miniconda3/envs/htd-mamba/lib/python3.10/site-packages (from triton==2.0.0->torch->thop==0.1.1.
post2209072238->-r requirements.txt (line 6)) (15.0.7)
Requirement already satisfied: MarkupSafe>=2.0 in /root/miniconda3/envs/htd-mamba/lib/python3.10/site-packages (from jinja2->torch->thop==0
.1.1.post2209072238->-r requirements.txt (line 6)) (3.0.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /root/miniconda3/envs/htd-mamba/lib/python3.10/site-packages (from sympy->torch->thop=
=0.1.1.post2209072238->-r requirements.txt (line 6)) (1.3.0)
Installing collected packages: torchsummary, tomli, threadpoolctl, six, pyparsing, numpy, kiwisolver, joblib, fonttools, einops, cycler, vc
s-versioning, scipy, python-dateutil, setuptools-scm, scikit-learn, matplotlib, thop
  Attempting uninstall: numpy
    Found existing installation: numpy 2.2.6
    Uninstalling numpy-2.2.6:
      Successfully uninstalled numpy-2.2.6
Successfully installed cycler-0.12.1 einops-0.8.0 fonttools-4.63.0 joblib-1.5.3 kiwisolver-1.5.0 matplotlib-3.5.0 numpy-1.26.4 pyparsing-3.
3.2 python-dateutil-2.9.0.post0 scikit-learn-1.5.0 scipy-1.15.3 setuptools-scm-10.2.0 six-1.17.0 thop-0.1.1.post2209072238 threadpoolctl-3.
6.0 tomli-2.4.1 torchsummary-1.5.1 vcs-versioning-2.2.2
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibl
y rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root
-user-action option if you know what you are doing and want to suppress this warning.
```

1.1 环境配置

编 selective_scan_cuda
export TORCH_CUDA_ARCH_LIST="7.0"

pip install . --no-build-isolation

```
(htd-mamba) root@autodl-container-97b911b700-920defa3:~/HTD-muban# pip install . --no-build-isolation
Collecting h11>=0.16 (from httpcore==1.*->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers->mamba_ssm==1.2.2)
  Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Collecting exceptiongroup>=1.0.2 (from anyio->httpx<1,>=0.23.0->huggingface-hub<2.0,>=1.5.0->transformers->mamba_ssm==1.2.2)
  Downloading exceptiongroup-1.3.1-py3-none-any.whl (16 kB)
Collecting shellingham>=1.3.0 (from typer->transformers->mamba_ssm==1.2.2)
  Downloading shellingham-1.5.4-py2.py3-none-any.whl (9.8 kB)
Collecting rich>=13.8.0 (from typer->transformers->mamba_ssm==1.2.2)
  Downloading rich-15.0.0-py3-none-any.whl (310 kB)
Collecting annotated-doc>=0.0.2 (from typer->transformers->mamba_ssm==1.2.2)
  Downloading annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Collecting markdown-it-py>=2.2.0 (from rich>=13.8.0->typer->transformers->mamba_ssm==1.2.2)
  Downloading markdown_it_py-4.2.0-py3-none-any.whl (91 kB)
Collecting pygments<3.0.0,>=2.13.0 (from rich>=13.8.0->typer->transformers->mamba_ssm==1.2.2)
  Downloading pygments-2.20.0-py3-none-any.whl (1.2 MB)
     ━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 1.2/1.2 MB 105.1 MB/s  0:00:00
Collecting mdurl~=0.1 (from markdown-it-py>=2.2.0->rich>=13.8.0->typer->transformers->mamba_ssm==1.2.2)
  Downloading mdurl-0.1.2-py3-none-any.whl (10.0 kB)
Building wheels for collected packages: mamba_ssm
  Building wheel for mamba_ssm (pyproject.toml) ... \
```

1.1 环境配置

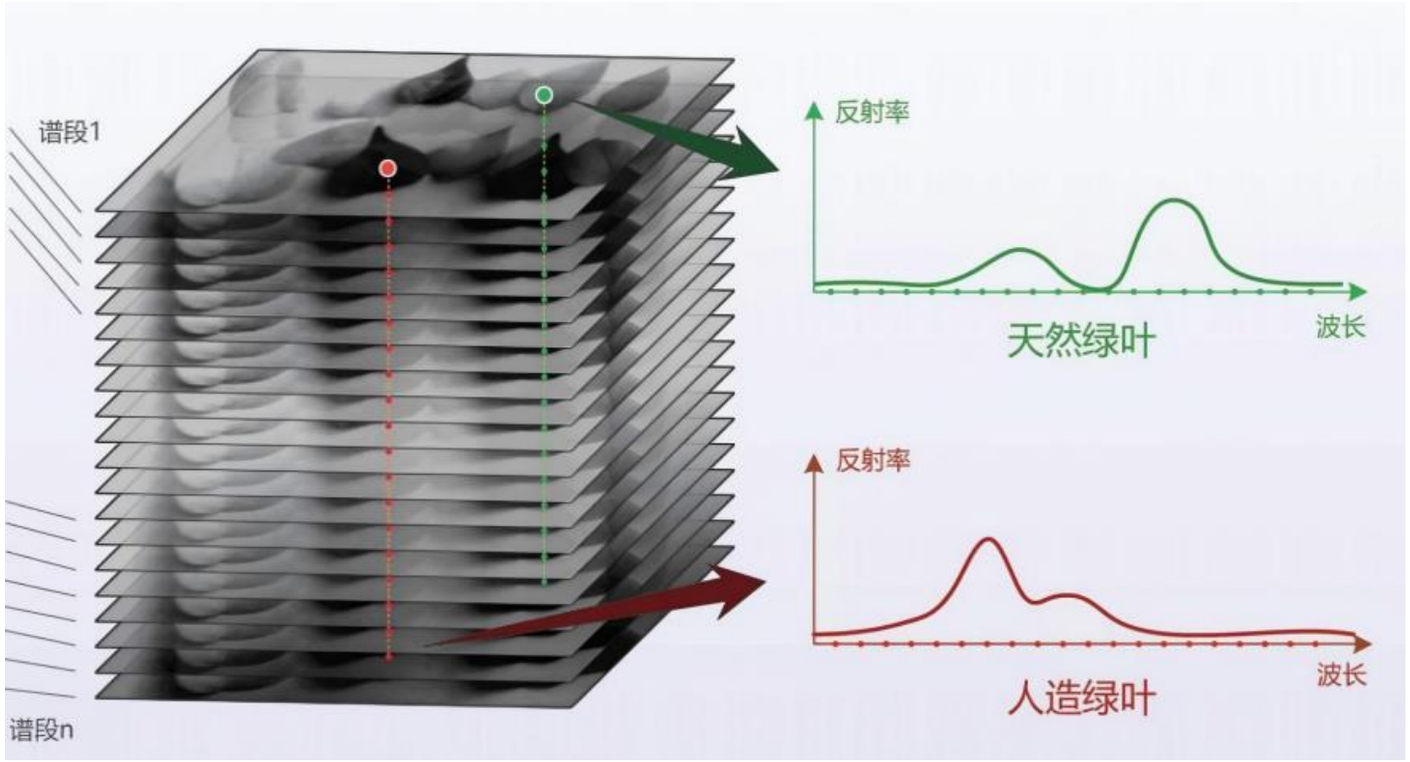
香 港 大 學
THE UNIVERSITY OF HONG KONG

装 causal_conv1d
export LD_LIBRARY_PATH=/root/miniconda3/envs/htd-mamba/lib/python3.10/site-packages/torch/lib:$LD_LIBRARY_PATH

pip install causal_conv1d==1.4.0 --no-build-isolation

```
(htd-mamba) root@autodl-container-97b911b700-920defa3:~/HTD-muban# pip install causal_conv1d==1.4.0 --no-build-isolation
v1d==1.4.0) (3.25.0)
Requirement already satisfied: lit in /root/miniconda3/envs/htd-mamba/lib/python3.10/site-packages (from triton==2.0.0->torch->causal_conv1
d==1.4.0) (15.0.7)
Requirement already satisfied: MarkupSafe>=2.0 in /root/miniconda3/envs/htd-mamba/lib/python3.10/site-packages (from jinja2->torch->causal_
conv1d==1.4.0) (3.0.3)
Requirement already satisfied: mpmath<1.4,>=1.1.0 in /root/miniconda3/envs/htd-mamba/lib/python3.10/site-packages (from sympy->torch->causa
l_conv1d==1.4.0) (1.3.0)
Building wheels for collected packages: causal_conv1d
  Building wheel for causal_conv1d (pyproject.toml) ... done
  Created wheel for causal_conv1d: filename=causal_conv1d-1.4.0-cp310-cp310-linux_x86_64.whl size=103586286 sha256=814b3c7b13316011192630f9
ee9fc17a3a129c4480c8c5eb8812b96bd242545b
  Stored in directory: /root/.cache/pip/wheels/d0/f4/db/b58cd28344f620200ac7c8a262c67ab6b2b1d9d18484f077fa
Successfully built causal_conv1d
Installing collected packages: causal_conv1d
Successfully installed causal_conv1d-1.4.0
WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibl
y rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root
-user-action option if you know what you are doing and want to suppress this warning.
```

1.2 数据介绍



在xy平面上，我们选择某一个像素点观察，获得的数据是那个像素点的光谱曲线，即该点在不同波段的反射率曲线，后续模型也是根据光谱曲线进行像素级目标识别。

传统RGB图像是三通道，高光谱数据立方体通常是几百通道。

高光谱数据没法像RGB图像直接查看，一般的查看方式是选择某一通道进行灰度成像，或者选择某三个通道进行伪彩色图像合成。

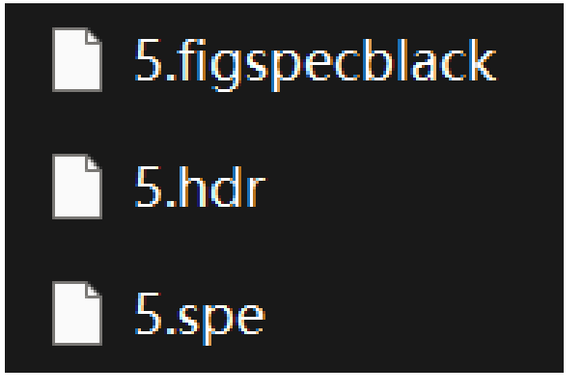
1.2 数据介绍

模型的训练是自监督训练，不需要人工标注数据。我们所做的数据标注是在测试阶段给模型提供正确答案，来检验模型的效果。

模型的输入数据是一个.mat格式的文件（由Matlab生成），其中包含两个变量，一个变量是data，存储的是原始的高光谱数据，高度×宽度×光谱波段；另一个变量是map，存储的目标的真实位置，也就是我们标注的数据。其是一个由0和1组成的矩阵，1的位置表示目标所在的真实位置，0表示的是背景位置。

高光谱目标检测相较于Yolo算法，呈现的结果不是标出框并且输出类别，而是实现像素级的目标检测，即告诉你哪些像素是目标像素，哪些是背景像素，检测效果更加精准。

1.2 数据预处理



5.hdr（头文件 / Header File）

作用：它是数据的“说明书”或“地图”。这是一个纯文本文件，里面记录了如何解读原始数据的具体参数，包括图像的宽度（大小）、高度（行数）、波段数（光谱维度）、数据类型、字节顺序以及数据存储的排列顺序（如 BSQ、BIL、BIP）等关键信息。

重要性：没有它，软件就无法正确识别和读取庞大的原始数据块。

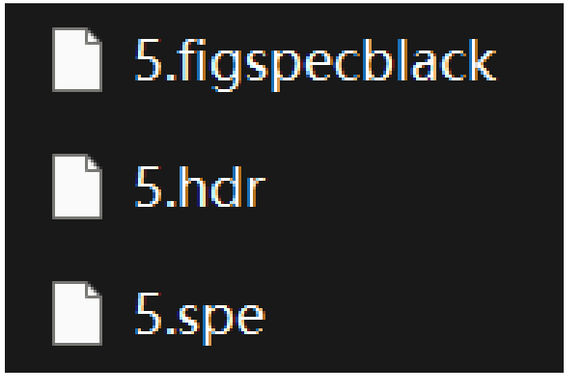
5.spe（光谱数据文件 / Spectral Data File）

作用：它是真正的“数据仓库”。这是一个二进制文件，体积通常较大，里面按顺序紧凑地存储了所有像素在各个波段下的灰度值或反射率值。

重要性：高光谱立方体（空间维度 × 光谱维度）的所有实际测量数值都包含在这个文件中。

1.2 数据预处理

香 港 大 學
THE UNIVERSITY OF HONG KONG



5.figspecblack（暗场/黑电平参考文件）

作用：它是用于数据校正的“基准参考图”。.figspec通常是特定高光谱成像系统（如 FigSpec 系列设备）的数据格式标识，而 black明确表示这是暗场数据。

重要性：在完全遮光（无光线进入）的条件下拍摄。它主要用于记录相机本身的暗电流噪声以及系统的背景基线。在后续的科学数据处理中，必须用它来消除传感器固有的热噪声和环境背景干扰。

1.2 数据预处理

高光谱数据本质上是一个三维立方体：二维的平面空间（行 × 列）加上一维的光谱波段。重建的过程，就是利用“说明书”来正确排列和解析“数据仓库”中的数据。

1.准备环境：确保安装了 numpy（用于矩阵运算）和 spectral（专门用于高光谱数据处理的库）等包。

2.解析头文件：编写代码读取 5.hdr文件，提取出空间维度的大小（samples和 lines）、波段数（bands）以及数据存储类型（data type）和排列方式（interleave）。

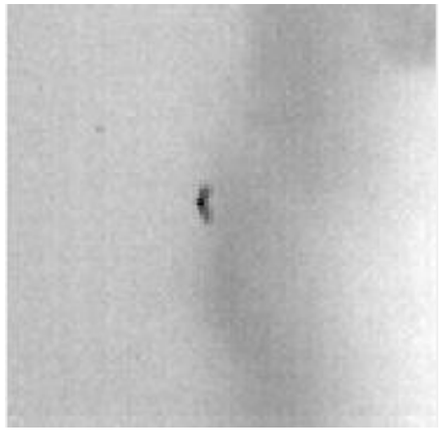
3.读取原始数据：以二进制模式（'rb'）打开 5.spe文件，根据头文件中指定的数据大小和类型，使用 numpy.fromfile()将其读取为一个一维数组。

4.重塑数据形态：高光谱立方体的重建关键一步是将一维数组按照正确的空间形状和光谱维度进行“重塑”。例如，如果头文件指明数据是 BIP（波段交织按像元存储）格式，则使用类似 data.reshape((lines, samples, bands))的代码将其转化为三维数组。

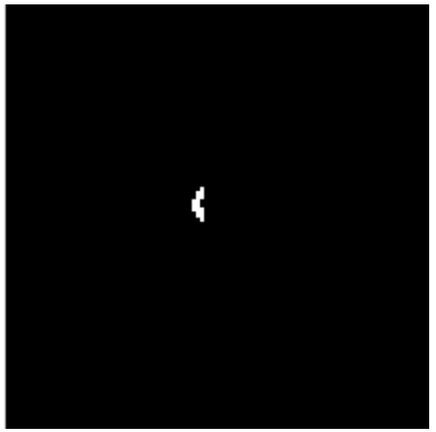
5.引入黑场校正（数据预处理）：在得到原始立方体后，读取 5.figspecblack文件并将其重塑为与原始数据相同的形状。最后，执行标准的暗场校正公式：校正后数据 = (原始数据 - 黑场数据) / (白板数据 - 黑场数据)（注：实际操作中通常还需要配合白板数据进行完整的相对反射率计算）。

1.3 数据标注

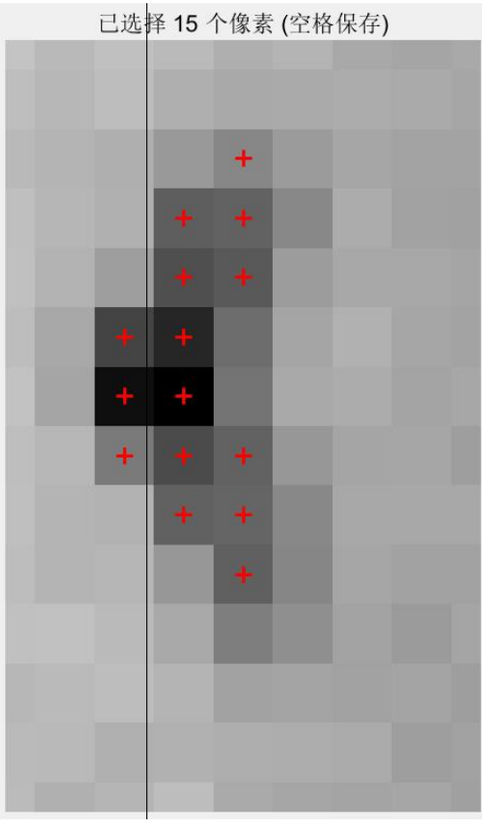
将处理好的原始数据读取，可以选择第一通道或者其它数值差异明显的通道进行成像，方便我们进行标注。接着我们可以放大该图像，然后用鼠标点击对应目标像素，然后保存为由0和1组成的矩阵，输出为.mat文件格式，变量空间命名为map。



第一通道成像



map文件



1.3 数据标注

使用Matlab或者Python，将data文件和map文件进行合并，组成一个双变量的.mat文件，模型通过读取data和map两个变量名进行读取相应的数据。

```
with torch.no_grad():
    mat = sio.loadmat(modelConfig["path"])
    data = mat['data']
    map = mat['map']
    data = standard(data)
    target_spectrum = ts_generation(data, map, 7)
    h, w, c = data.shape
```

1.3 数据标注

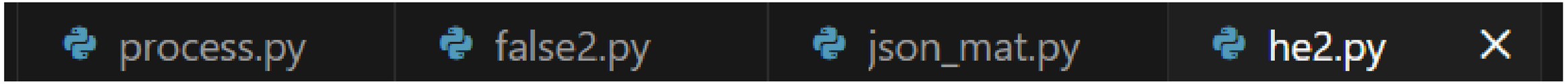
process代码，将原始高光谱数据重塑为高光谱立方体。

false代码，将高光谱某一通道数据进行成像，利于后面标注。

接下来在本地电脑安装labelme（pip install labelme），服务器没有图形化可视化后端

json代码，将labelme标注好的数据转换为mat格式

he代码，将数据整理成模型的规范输入形式。



2026

目录

CONTENTS

01 | 数据介绍与预处理

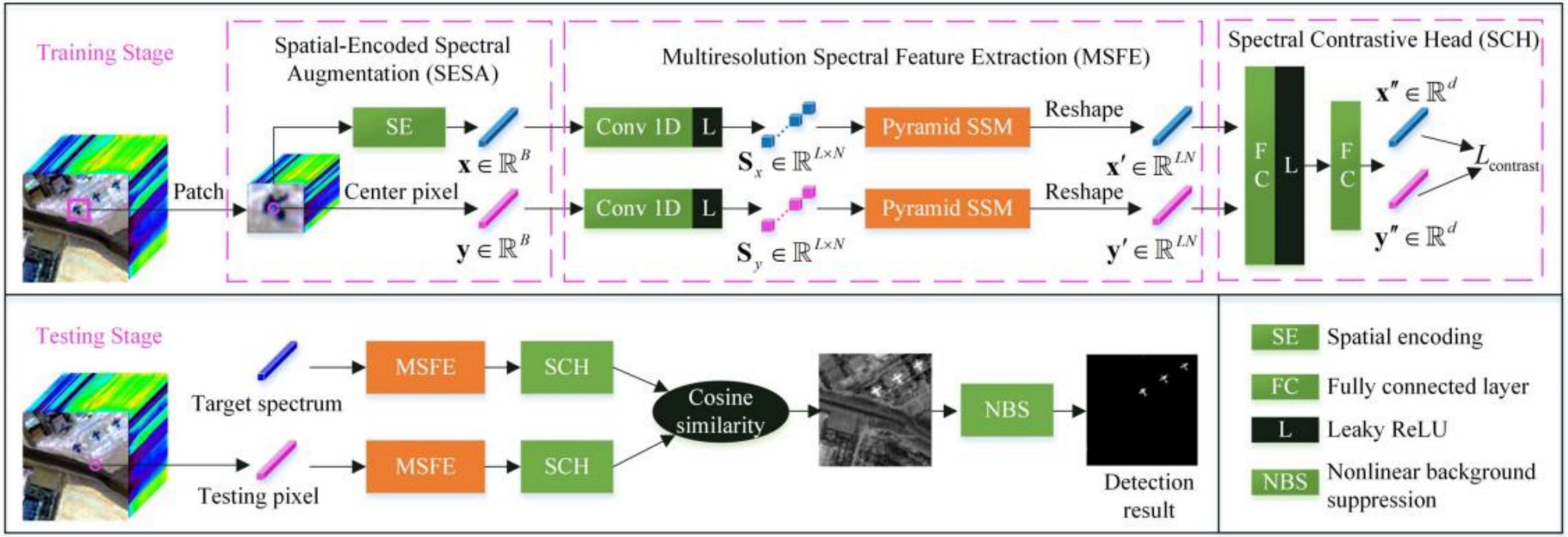
02 | 模型介绍与代码讲解

03 | 实践与模型优化

2.1 模型介绍

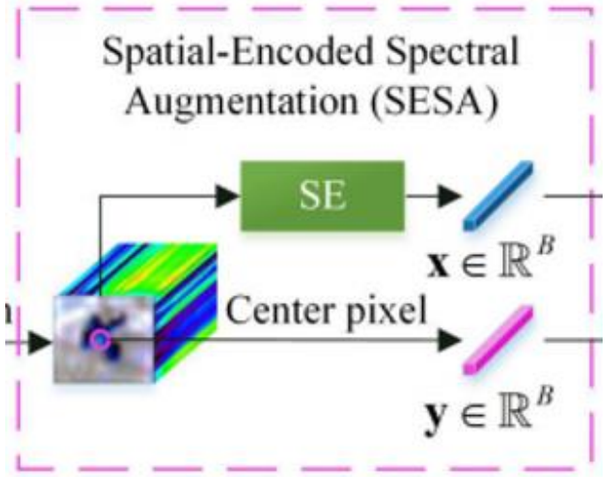
香 港 大 學
THE UNIVERSITY OF HONG KONG

模型架构图

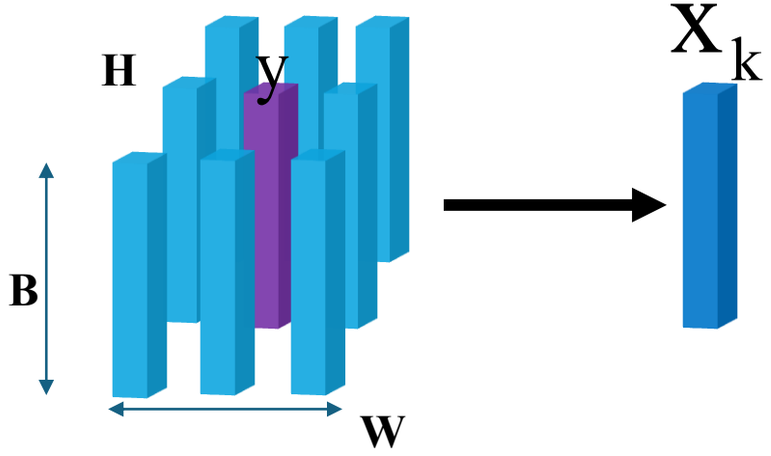


2.1 模型介绍

香 港 大 學
THE UNIVERSITY OF HONG KONG



SESA模块



$$\mathbf{x_k} = \mathbf{P_k^T W}$$

$$\mathbf{W}_i = \frac{\exp\left(c\left(\mathbf{y}, \mathbf{P}_i\right)\right)}{\sum_{j=1}^{p^2} \exp\left(c\left(\mathbf{y}, \mathbf{P}_j\right)\right)}$$

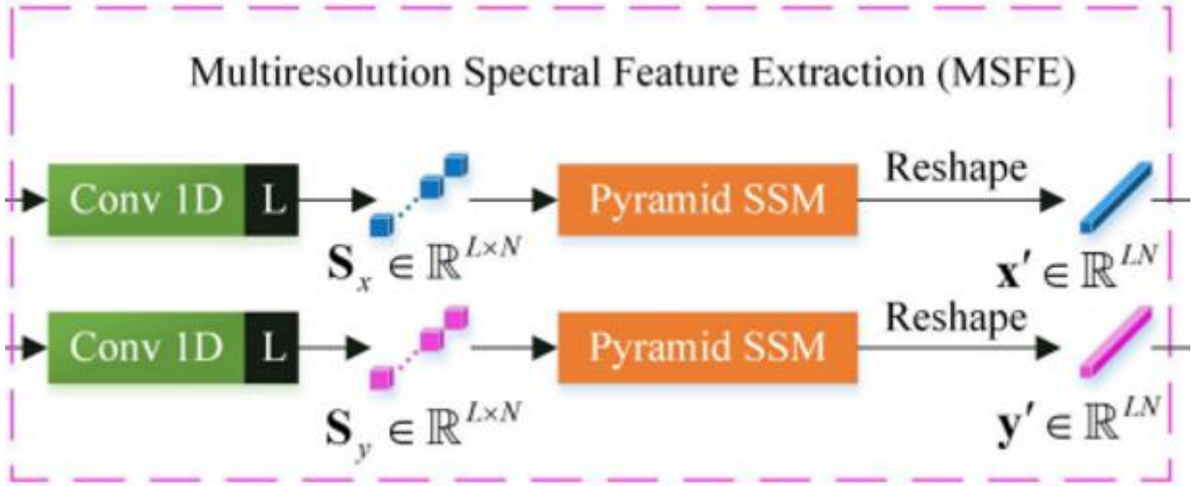
$$c\left(\mathbf{a}, \mathbf{b}\right) = \left(\mathbf{a}^{\mathrm{T}}\mathbf{b}\right) / \left(\|\mathbf{a}\|_2 \cdot \|\mathbf{b}\|_2\right)$$

给定一个随机图像块 $\mathbf{P} \in \square^{p^2 \times B}$（其中为块大小，为波段数），其中心像素原始光谱为 $\mathbf{y} \in \square^{B}$。通过计算块内所有像素与中心像素的光谱相似度作为权重，可将邻域的空间信息编码，生成中心像素的增强光谱视图 $\mathbf{X_k}$（新像素）。

2.1 模型介绍

香 港 大 學
THE UNIVERSITY OF HONG KONG

MSFE模块

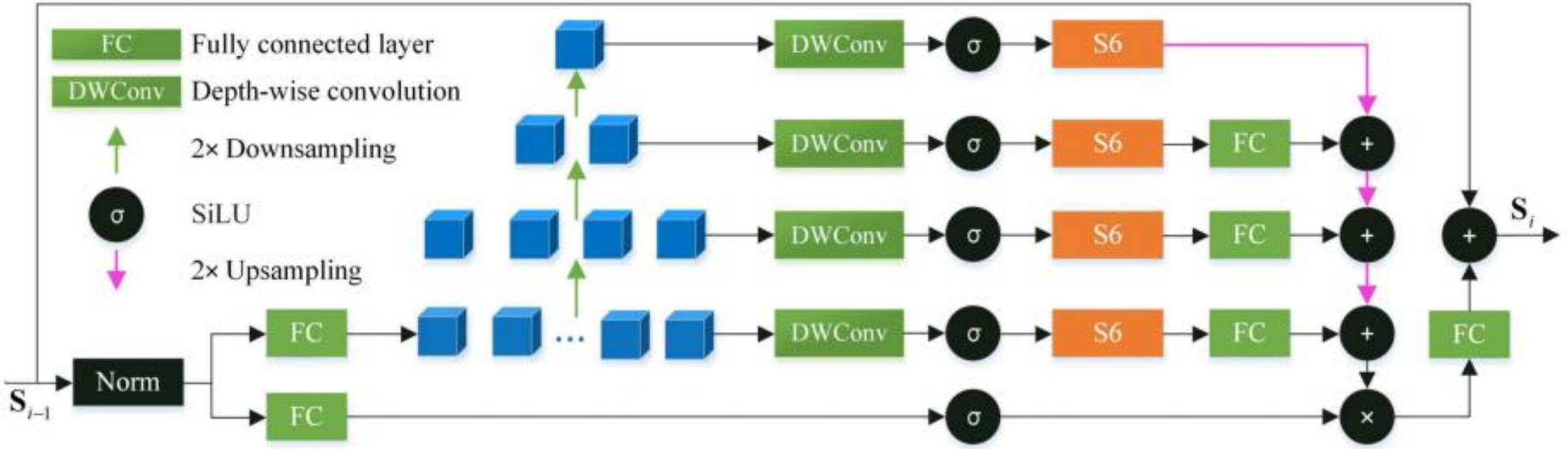


多光谱数据的相邻波段往往高度相关且存在信息冗余，直接使用可能降低特征提取效率。为此，我们采用一个可学习的一维卷积模块将输入光谱转换为分组令牌（token）。具体做法是，通过一维卷积以一定步长沿光谱维滑动，提取局部特征并将整个光谱划分为若干组。

2.1 模型介绍

香 港 大 學
THE UNIVERSITY OF HONG KONG

Pyramid SSM

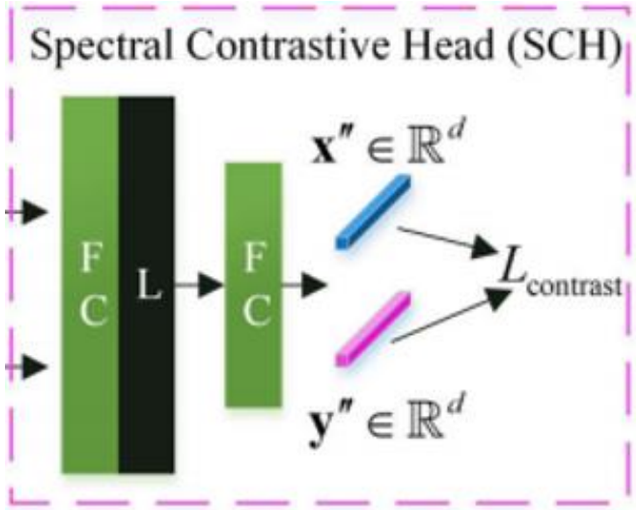


卷积过后的组级光谱嵌入可以看作一个光谱token序列,除了用单尺度Mamba捕捉序列的长程依赖特征，其多尺度的特征分析也很重需要,因此使用了一种多尺度Mamba网络，用来提取不同分辨率下的光谱本质特征。

2.1 模型介绍

香 港 大 學
THE UNIVERSITY OF HONG KONG

SCH模块

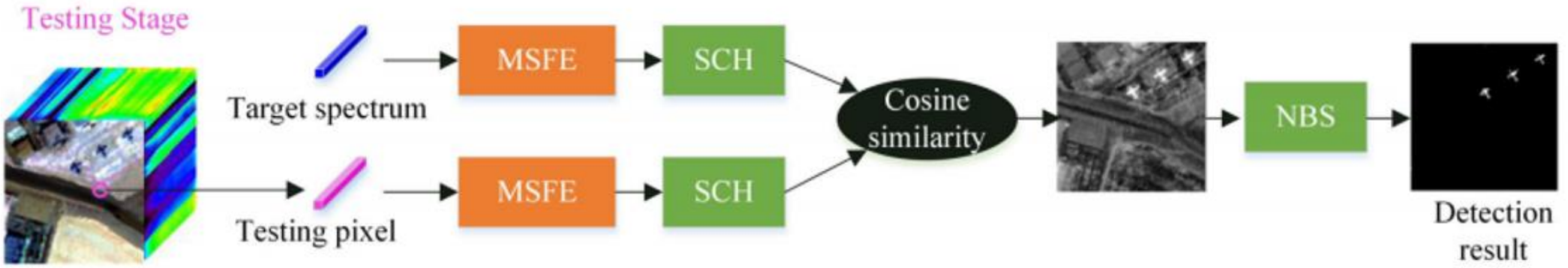


通过一个两层感知机，进一步将提取的多分辨率光谱特征投影到一个适于进行相似度对比的低维特征空间。用对比学习的方式训练模型。本模型里同一点上得到的特征向量对构成了正样本对，不同点上的特征向量对构成了负样本对。

2.1 模型介绍

香 港 大 學
THE UNIVERSITY OF HONG KONG

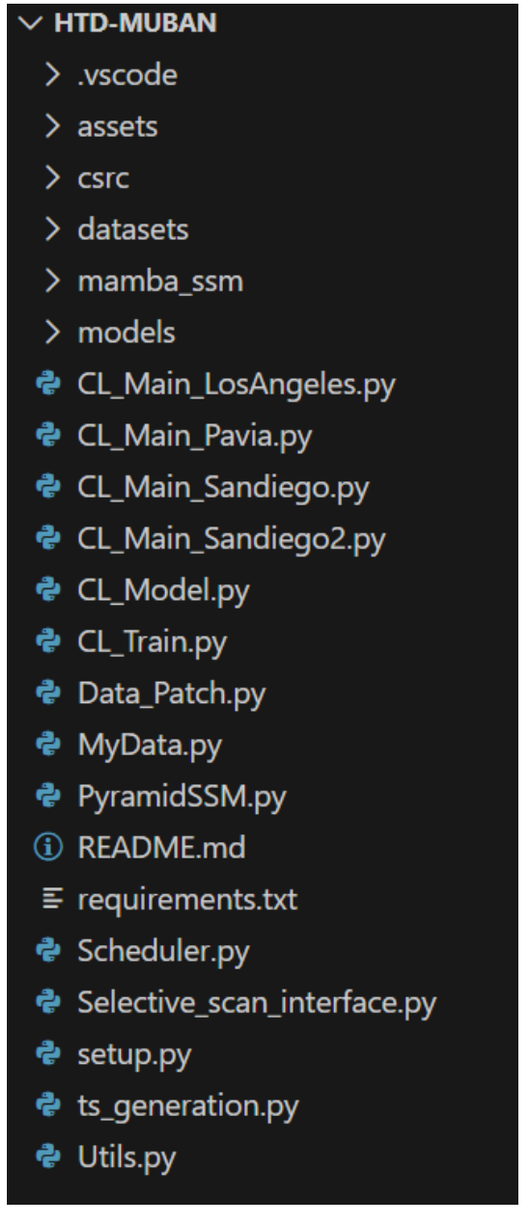
测试流程



每个待检测像素 Xi 与已知的目标先验光谱 d 配对，并一同通过训练好的多尺度光谱增强模块和特征提取主干网络，得到它们的特征表示。通过计算二者特征的余弦相似度，得到该像素的初始检测得分。然后利用一个非线性背景抑制函数对得分图进行优化，得到最终的检测结果。

$$\mu_i = \frac{f(\mathbf{X}_i)^T f(\mathbf{d})}{\|f(\mathbf{X}_i)\|_2 \cdot \|f(\mathbf{d})\|_2} \qquad \mu_i^{'} = \exp\left(-\frac{(\mu_i - 1)^2}{\delta}\right)$$

2.2 代码讲解



模型代码位置说明：

CL_Main开头的四个文件，是原文提供的数据集验证示例。

CL_Model文件实现了模型的搭建。

CL_Train文件包含了模型训练、寻找最优权重以及测试的代码。

Data_Patch文件，设计一些数据标准处理代码，以及数据增强模块的代码，即构建正负样本对部分。

PyramidSSM是论文中金字塔SSM结构的实现。

ts_generation是常见的高光谱目标光谱提取方法，在标注的光谱中计算出一个更具表征的目标光谱，即测试目标。

剩下的一些文件是Mamba模块的实现基础，了解即可，本实践中直接调用Mamba模块。

2.2 代码讲解

香 港 大 學
THE UNIVERSITY OF HONG KONG

CL_Model.py

```python
class SpectralGroupAttention(nn.Module):
    def __init__(self, band=189, group_length=20, channel_dim=128, state_size=128, device=None, layer=1):
        super().__init__()
        stride = group_length // 4
        self.group_division = nn.Sequential(
            nn.Conv1d(1, channel_dim, group_length, stride=stride, padding=0),
            nn.LeakyReLU()
        )
        sequence_length = (band - group_length) // stride + 1
        self.mamba = Mamba(seq_len=sequence_length, d_model=channel_dim, state_size=state_size,
                           layer=layer)
        self.fnn = nn.Sequential(
            nn.Linear(sequence_length * channel_dim, 64),
            nn.LeakyReLU(),
            nn.Linear(64, 32)
        )

    def forward(self, x):
        x = self.group_division(x)
        x = torch.permute(x, (0, 2, 1))
        x = self.mamba(x)
        x = x.view([x.shape[0], -1])
        x = self.fnn(x)
        return x
```

这是整个检测模型的"前端"定义文件，核心类是 SpectralGroupAttention。它把一条高光谱像素（band 维，比如 189 个波段）先用一个 1D 卷积按 group_length滑窗切分成若干"光谱组"，stride 取 group_length//4 让组之间有重叠，卷积输出通道 channel_dim就是每组压成的特征维；接着把维度 permute 成 (B, seq_len, channel_dim) 喂给自定义的 Mamba（来自 PyramidSSM），让状态空间模型沿着"光谱组序列"方向建模长程依赖；Mamba 出来后 flatten 接两层 MLP（64→32）得到最终 32 维特征。__main__里还顺手用 torchsummary 和 thop 把参数量、FLOPs 打了出来，方便调结构时看复杂度

2.2 代码讲解

CL_Train.py

```python
def train(modelConfig: Dict):
    seed_torch(modelConfig['seed'])
    device = torch.device(modelConfig["device"])
    dataset = Data(modelConfig["path"], w_size=modelConfig['patch
    dataloader = DataLoader(dataset, batch_size=modelConfig["batc
                            pin_memory=True)
    # model setup
    net_model = SpectralGroupAttention(band=modelConfig['band'],
                                       channel_dim=modelConfig['c
```

```python
def select_best(modelConfig: Dict):
    seed_torch(modelConfig['seed'])
    device = torch.device(modelConfig["device"])
    opt_epoch = 0
    max_auc = 0
    path = modelConfig["save_dir"] + '/' + modelConfig['dataset'] + '/'
    for e in range(0, modelConfig['epoch'], 1):
        with torch.no_grad():
            mat = sio.loadmat(modelConfig["path"])
            data = mat['data']
            map = mat['map']
```

```python
def eval(modelConfig: Dict):
    start = time.perf_counter()
    seed_torch(modelConfig['seed'])
    device = torch.device(modelConfig["device"])
    path = modelConfig["save_dir"] + '/' + modelConfig['dataset'] + '/'
    with torch.no_grad():
        mat = sio.loadmat(modelConfig["path"])
        data = mat['data']
        map = mat['map']
        data = standard(data)
        target_spectrum = ts_generation(data, map, 7)
        h, w, c = data.shape
```

这是训练和测试的主流程文件，任务是"高光谱目标检测"，但训练用的是对比学习。train()里先读 HSI 数据包装成 Data的 DataLoader，每个样本返回一个中心像素 + 一个 patch 编码向量（来自 Data_Patch 的余弦加权），然后把这两类拼一起过 SpectralGroupAttention，用 info_nce_loss做自监督——同一像素的 center 和 coded 是正对，其余互为负对，温度系数 0.1，本质是让模型学会"中心像素"和"它所在 patch 的上下文编码"在特征空间对齐。select_best()和 eval()是下游检测：拿 ts_generation产出的目标先验光谱过模型得 target_feature，再逐 batch 把整幅图的每个像素过模型，算 cosine similarity 得到检测图，最后用 ROC/AUC（还多了几个变种指标 auc2~auc5）评价。paintTrend画 loss 曲线，normalize/cosin_similarity是配套小工具。调度器用了 warmup + CosineAnnealing（Scheduler.py里的 GradualWarmupScheduler）

2.2 代码讲解

香 港 大 學
THE UNIVERSITY OF HONG KONG

PyramidSSM.py

```python
class SSM_Manipulation(nn.Module):
    '''
    The ssm manipulation to capture the long-range dependencies
    '''

    def __init__(self, d_model, state_size, d_inner, d_conv, conv_bias=True, use_casual1D=True, adjust=True):
        super(SSM_Manipulation, self).__init__()
        self.conv1d = nn.Conv1d(
            in_channels=d_inner,
            out_channels=d_inner,
            bias=conv_bias,
            kernel_size=d_conv,
            groups=d_inner,
            padding=d_conv - 1
        )  # 2N->2N, depth-wise convolution, L+d_conv-1
        self.ssm = S6(d_model=d_model, d_state=state_size, d_inner=d_inner)
        self.activation = "silu"  # y=x*sigmoid(x)
        self.act = nn.SiLU()
        self.use_casual1D = use_casual1D
        if adjust:
            self.adjust = nn.Conv1d(d_inner, d_inner, kernel_size=1)
```

这是论文最核心的"金字塔 Mamba"模块，相当于把 Mamba 改造成 U 形多尺度。顶层 Mamba类可以叠多层 MambaBlock；每个 Block 里先把输入做 RMSNorm，再走一个 in_proj（线性扩 2 倍）拆成 x 和 gating 的 z，x 走四条支路：原尺度 ssm_x1、downsample 一次走 ssm_x2、downsample 两次走 ssm_x3、downsample 三次走 ssm_x4（最深处 adjust=False不再做 1×1 调整），然后对称地 upsample 回来加 skip connection（x3_sup = up1(x4_ssm)+x3_ssm 这种），最后乘 silu(z) 再 out_proj回原 dim 加残差。SSM_Manipulation包了一个 depthwise causal conv1d（用 causal_conv1d_fn加速）+ 真正的 S6 扫描，S6类里把 Δ/B/C 的投影、A 的 HiPPO 初始化、softplus 约束 Δ 都写全了，forward 默认走 selective_scan_fn（CUDA 版），后备才走纯 PyTorch 的 for-loop 扫描。DownSample/UpSample就是带 stride 2 的 1D conv / transposed conv，序列长度减半/加倍用的

2.2 代码讲解

Data_Patch.py

```
def __getitem__(self, index):
    position_y = index // self.h
    position_x = index - position_y * self.h
    position_x = position_x + self.pad_size
    position_y = position_y + self.pad_size
    windows_out = self.data[position_x - self.pad_size:position_x + self.pad_size + 1,
                            position_y - self.pad_size:position_y + self.pad_size + 1, :]
    center = windows_out[self.pad_size, self.pad_size]
    center = center[None, :]
    patch = windows_out
    coded_vector = patch_encoded(patch, center)
    return torch.from_numpy(center).float(), torch.from_numpy(np.array(coded_vector)).float()
```

高光谱数据的 Dataset 封装。Data类读 .mat里的 data（H×W×B），全局 min-max standard 后四周 reflect pad 出 w_size//2的边，按像素索引切出 w_size×w_size的 patch；__getitem__返回两个东西：中心像素 center（1×B）和 coded_vector（1×B）——后者是 patch_encoded算的，把 patch 里所有像素和 center 算 cosine similarity 做 softmax 权重，再加权平均得到一条"上下文编码谱"，这就是对比学习里和 center 配对的那个 view。cosin_similarity/standard顺手也定义了，后者和 Utils 里重了。

2.2 代码讲解

ts_generation.py

```python
def ts_generation(data, gt, type=0):
    '''
    using different methods to select the target spectrum
    :param type:
    0-Averaged spectrum
    1-Averaged spectrum after spatial debiasing (L2 norm)
    2-Averaged spectrum after spectral debiasing (spectral angle)
    3-Spatial median spectrum (median of L2 norm)
    4-Spectral median spectrum (median of spectral angles)
    5-The target pixel with the closest Euclidean distance to all the other target pixels
    6-The target pixel with the closest cosine distance to all the target pixels
    7-The target pixel closest to the averaged spectrum (L2 norm)
    8-The target pixel closest to the averaged spectrum (spectral angle)
    :return:
    '''
    ind = np.where(gt == 1)
```

目标先验光谱的 9 种生成策略，由 type参数选。type=0直接所有标注目标像素平均；1/2 是在平均谱基础上按 L2 距离或光谱角筛掉最远的 20% 再做平均（去离群）；3/4 是空间中值/光谱中值（排序取中间）；5/6 是找"和其他所有目标像素平均距离最小"的那个真实像素（类 medoid）；7/8 是找"离平均谱最近"的真实像素（论文 CL_Train里默认用的 type=7，即 L2 最近那条）。输出都是 B×1，后面会 target_prior.unsqueeze(1) 变成 1×1×B 喂模型

2.2 代码讲解

Scheduler.py —— 就一个类 GradualWarmupScheduler，继承 _LRScheduler。构造时传 multiplier（warmup 末倍率）、warm_epoch（warmup 轮数）、after_scheduler（这里挂 CosineAnnealingLR）。逻辑是 last_epoch <= warm_epoch时 LR 从 base_lr 线性爬到 base_lr × multiplier，之后把 after_scheduler.base_lrs乘上 multiplier 交棒给余弦退火。step()里区分"还在 warmup"和"已交棒"两种情况。

Utils.py —— 俩工具函数：standard是 min-max 归一化到 [0,1]，防除零；checkFile是 mkdir -p 封装，训练保存 ckpt 和输出图时用。

2.3 如何训练并测试自己的数据集

香 港 大 學
THE UNIVERSITY OF HONG KONG

```python
def main(model_config=None):
    # 针对您1024x1024x16数据集的配置
    modelConfig = {
        "state": "eval",  # 首次运行请先设为"train"进行训练，训练完成后可设为"select_best"选择最佳模型，最后设为"eval"进行
        "epoch": 60,      # 训练轮数，根据数据量较大可考虑适当减少
        "band": 16,         # 光谱波段数，您的数据为16
        "batch_size": 80,  # 批大小，根据GPU内存可调整
        "seed": 1,          # 随机种子
        "channel": 16,      # 嵌入维度N，论文中设为16
        "lr": 1e-4,         # 学习率
        "multiplier": 2.,  # 学习率预热参数
        "epision": 10,      # AUC值小数位数
        "grad_clip": 1.,    # 梯度裁剪
        "device": "cuda:0",  # 使用GPU
        "training_load_weight": None,  # 训练时加载的权重，None表示从头训练
        "save_dir": "./models",        # 模型保存目录
        "test_load_weight": "ckpt_8_.pt",  # 测试时加载的权重，训练完成后会自动生成
        "patch_size": 3,    # 论文参数分析表明，对于目标较小的场景，较小的patch size更合适
        "m": 2,             # 光谱分组长度，由于您只有16个波段，设置为2以获得合适的序列长度
        "state_size": 16,  # SSM隐藏状态大小
        "layer": 1,         # 网络深度，论文中设为1以保持轻量
        "delta": 0.1,       # 背景抑制参数
        "dataset": "MyData",  # 您的数据集名称
        "path": "datasets/MyData.mat"  # 您的数据路径，请确保.mat文件包含'data'和'map'变量
    }
```

新建一个python文件，进行模型的参数预设，运行时直接python+此文件名.py。

state部分，进行选择是训练模型还是测试模型，模型的相关参数也是在下方调整。

2026

目录

CONTENTS

01 | 数据介绍与预处理

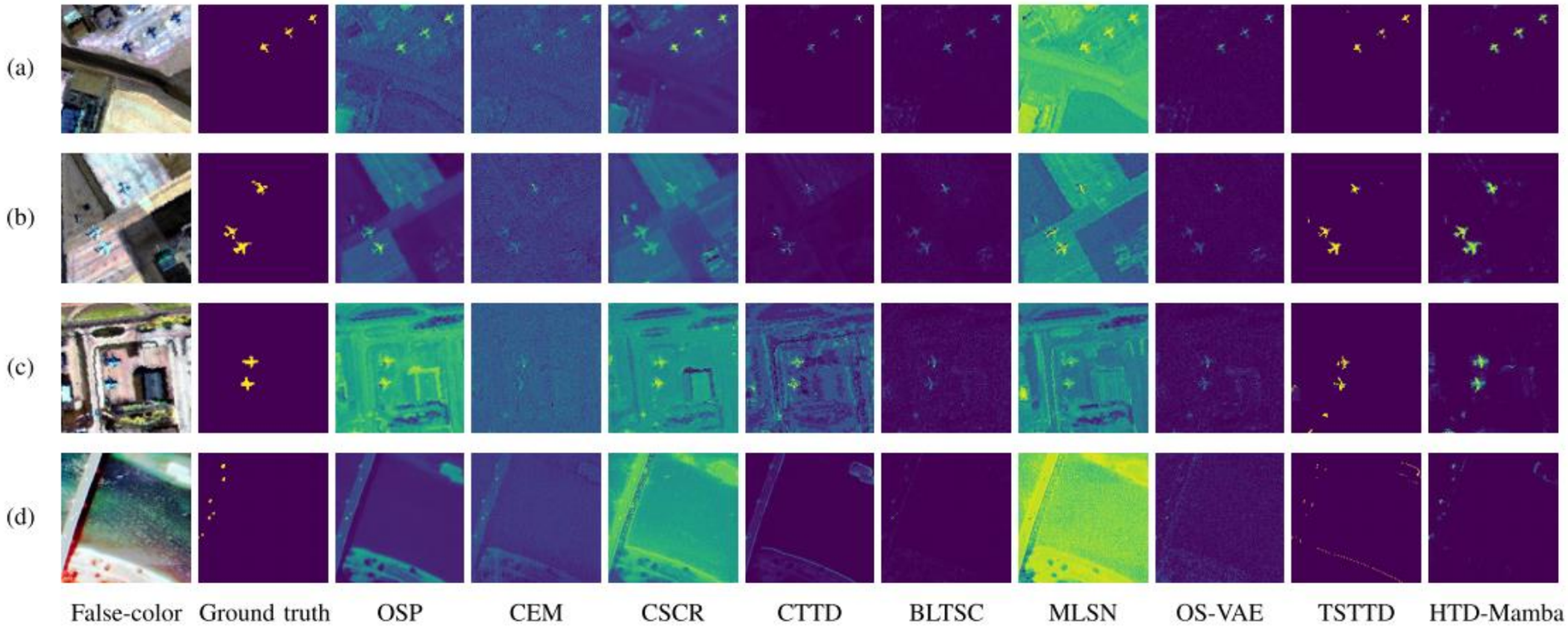
02 | 模型介绍与代码讲解

03 | 实践与模型优化

3.1 结果分析

香 港 大 學
THE UNIVERSITY OF HONG KONG

定性分析



概率热力图可以清楚地表现出模型的定性检测结果，把每一个像素上被判定为目标的概率用色彩来表现出来。像素颜色越接近亮黄色表示是目标的概率越高，越接近深蓝色则代表是背景的概率更高。理想的状况是目标所在的区域应该有明显的高亮、集中的黄色，背景区域全部变成蓝色。热力图质量好，就是说模型在不同的检测阈值下可以同时达到高的检出率和低的虚警率，所以它的整体适用性也更好。

3.1 结果分析

香 港 大 學
THE UNIVERSITY OF HONG KONG

定量指标

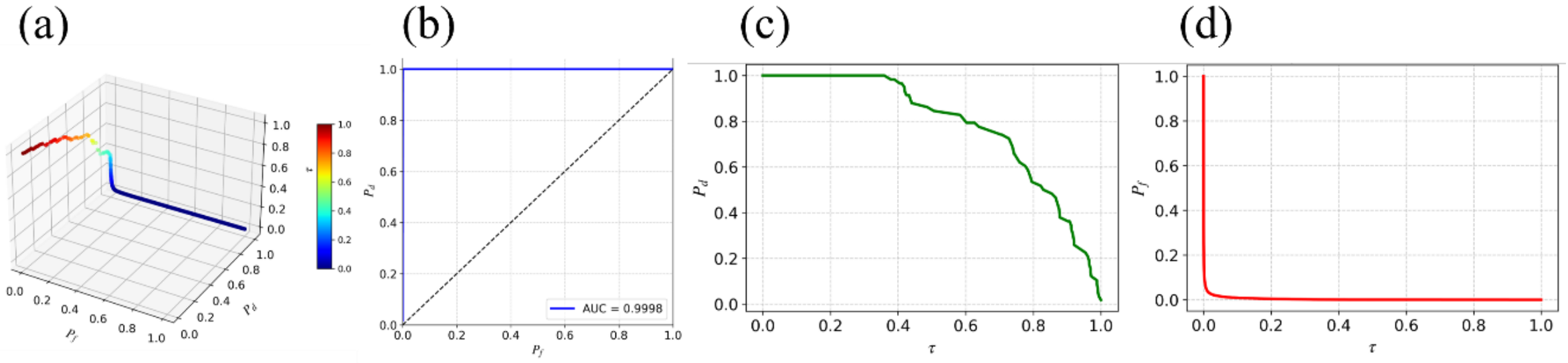
指标	$AUC_{(P_f,P_d)}$	$AUC_{(\tau,P_d)}$	$AUC_{(\tau,P_f)}$	AUC_{OA}	AUC_{SNPR}
特性	检测有效性	检测能力	虚警	整体性能	整体性能
范围	$[0,1]$	$[0,1]$	$[0,1]$	$[-1,2]$	$[0,+\infty]$
趋势	↑	↑	↓	↑	↑
最佳值	1	1	0	2	$+\infty$

定量评价就是计算ROC曲线与坐标轴围成的面积，对模型进行定量评估。具体来说，ROC曲线是不同阈值下，虚警率和检测概率的函数。为了综合分析，采用了三维ROC曲线，并从中获得三条展开的二维ROC曲线用来计算检测有效性指标、检测能力指标、虚警控制指标。其中，P_d表示的是识别正确的目标像素占全部像素的比例，P_f是识别错误的目标像素占全部像素的比例。模型最终输出的结果是每个像素是目标的概率值，如果其高于阈值则判定为目标像素，反之则为背景像素。以目标检测有效性指标为例，其是以为横轴，为纵轴，计算函数所围成的面积，以评价不同阈值下模型的检测有效性。

3.1 结果分析

香 港 大 學
THE UNIVERSITY OF HONG KONG

定量指标



ROC曲线示意图。（a）为ROC三维曲线示意图,（b）为检测有效性指标ROC二维曲线示意图,（c）为检测能力指标ROC二维曲线示意图,（d）为虚警控制指标ROC二维曲线示意图。

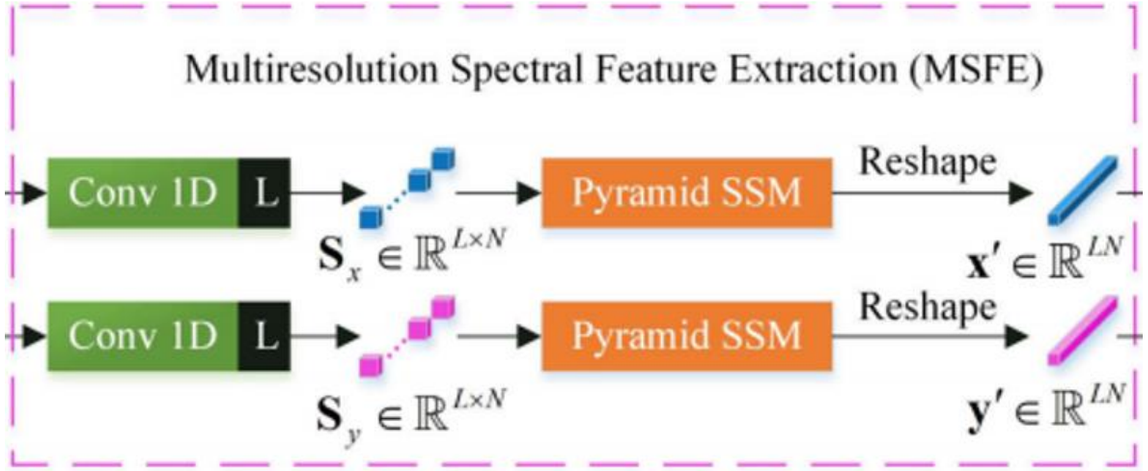
3.2 模型参数优化

香 港 大 學
THE UNIVERSITY OF HONG KONG

```python
def main(model_config=None):
    # 针对您1024x1024x16数据集的配置
    modelConfig = {
        "state": "eval",  # 首次运行请先设为"train"进行训练，训练完成后可设为"select_best"选择最佳模型，最后设为"eval"进行
        "epoch": 60,      # 训练轮数，根据数据量较大可考虑适当减少
        "band": 16,        # 光谱波段数，您的数据为16
        "batch_size": 80,  # 批大小，根据GPU内存可调整
        "seed": 1,         # 随机种子
        "channel": 16,     # 嵌入维度N，论文中设为16
        "lr": 1e-4,        # 学习率
        "multiplier": 2.,  # 学习率预热参数
        "epision": 10,     # AUC值小数位数
        "grad_clip": 1.,   # 梯度裁剪
        "device": "cuda:0",  # 使用GPU
        "training_load_weight": None,  # 训练时加载的权重，None表示从头训练
        "save_dir": "./models",       # 模型保存目录
        "test_load_weight": "ckpt_8_.pt",  # 测试时加载的权重，训练完成后会自动生成
        "patch_size": 3,   # 论文参数分析表明，对于目标较小的场景，较小的patch size更合适
        "m": 2,            # 光谱分组长度，由于您只有16个波段，设置为2以获得合适的序列长度
        "state_size": 16,  # SSM隐藏状态大小
        "layer": 1,        # 网络深度，论文中设为1以保持轻量
        "delta": 0.1,      # 背景抑制参数
        "dataset": "MyData",  # 您的数据集名称
        "path": "datasets/MyData.mat"  # 您的数据路径，请确保.mat文件包含'data'和'map'变量
    }
```

1.group_length m 调小 + N 适当调大：建议 m 从默认的 30/15/5 往下调，比如 3 或 5（视总波段数），stride 仍取 m//4 保重叠。这样 L 变长，Mamba 能吃到更细粒度的光谱变化。CL_Model.py里SpectralGroupAttention的 group_length和 channel_dim跟着调，channel_dim建议从 16 提到 32（论文 N=16 是全数据集通用，小目标需要更强表征）

3.2 模型参数优化



HTD-Mamba模型处理的本质，是将光谱信息视作光谱序列（可达1200通道），用Mamba处理长序列具有计算复杂度小、长程建模能力强的优势。但是Mamba在处理局部信息能力不足，而CNN的局部特征能力突出，所以采用CNN+Mamba的组合。即在模型输入时，先进行CNN进行局部特征提取，再用Mamba构建长程依赖。其中，步幅m的大小决定CNN一次卷积所涉及的通道数，实验表明当检测目标的目标像素数量较小时，较小的步幅会有较好的检测效果。

3.2 模型参数优化


```python
def eval(modelConfig: Dict):
    start = time.perf_counter()
    seed_torch(modelConfig['seed'])
    device = torch.device(modelConfig["device"])
    path = modelConfig["save_dir"] + '/' + modelConfig['dataset'] + '/'
    with torch.no_grad():
        mat = sio.loadmat(modelConfig["path"])
        data = mat['data']
        map = mat['map']
        data = standard(data)
        target_spectrum = ts_generation(data, map, 7)
        h, w, c = data.shape
```

2.ts_generation 换 type：模型提供了多种目标光谱提取的方式，模型默认采取的是type=7，可以尝试使用其它方法。不同的光谱提取方法，在不同的场景下表现不同，可以多做尝试。

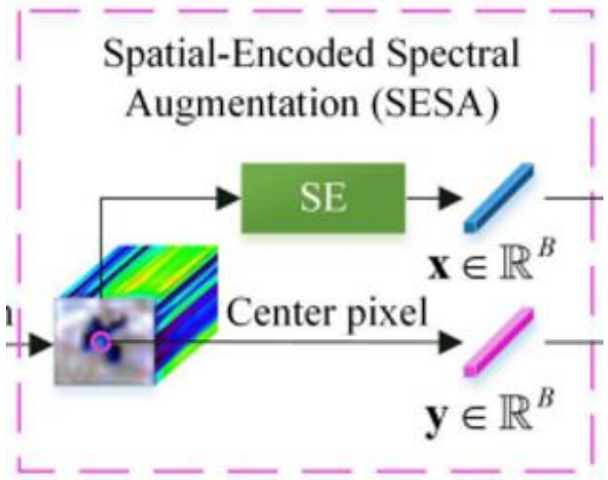
3.2 模型参数优化

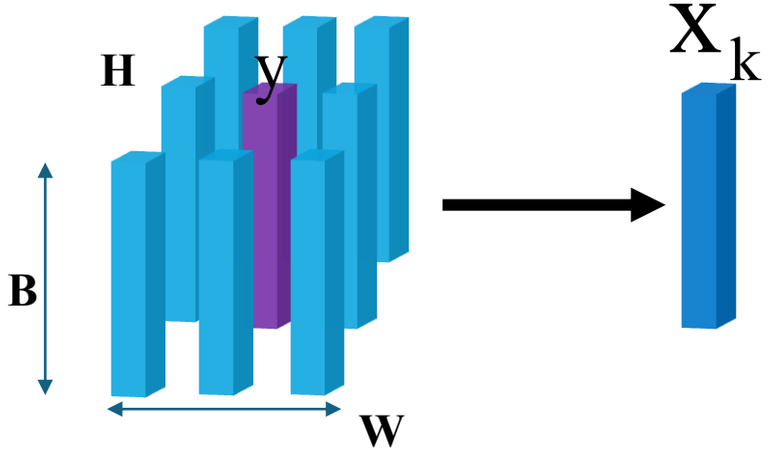
香 港 大 學
THE UNIVERSITY OF HONG KONG

```python
def main(model_config=None):
    # 针对您1024x1024x16数据集的配置
    modelConfig = {
        "state": "eval",  # 首次运行请先设为"train"进行训练，训练完成后可设为"select_best"选择最佳模型，最后设为"eval"进行
        "epoch": 60,      # 训练轮数，根据数据量较大可考虑适当减少
        "band": 16,        # 光谱波段数，您的数据为16
        "batch_size": 80,  # 批大小，根据GPU内存可调整
        "seed": 1,         # 随机种子
        "channel": 16,     # 嵌入维度N，论文中设为16
        "lr": 1e-4,        # 学习率
        "multiplier": 2.,  # 学习率预热参数
        "epision": 10,     # AUC值小数位数
        "grad_clip": 1.,   # 梯度裁剪
        "device": "cuda:0",  # 使用GPU
        "training_load_weight": None,  # 训练时加载的权重，None表示从头训练
        "save_dir": "./models",       # 模型保存目录
        "test_load_weight": "ckpt_8_.pt",  # 测试时加载的权重，训练完成后会自动生成
        "patch_size": 3,    # 论文参数分析表明，对于目标较小的场景，较小的patch size更合适
        "m": 2,            # 光谱分组长度，由于您只有16个波段，设置为2以获得合适的序列长度
        "state_size": 16,  # SSM隐藏状态大小
        "layer": 1,        # 网络深度，论文中设为1以保持轻量
        "delta": 0.1,      # 背景抑制参数
        "dataset": "MyData",  # 您的数据集名称
        "path": "datasets/MyData.mat"  # 您的数据路径，请确保.mat文件包含'data'和'map'变量
    }
```

3.patch_size 参数调整：G2A 小目标 w_size 建议调整成3，5或者7。太大会引入较大的噪声。具体的可以进行尝试和调整。

3.2 模型参数优化





$$\mathbf{x_k} = \mathbf{P_k^T W}$$

$$\mathbf{W}_i = \frac{\exp\left(c\left(\mathbf{y}, \mathbf{P}_i\right)\right)}{\sum_{j=1}^{p^2} \exp\left(c\left(\mathbf{y}, \mathbf{P}_j\right)\right)}$$

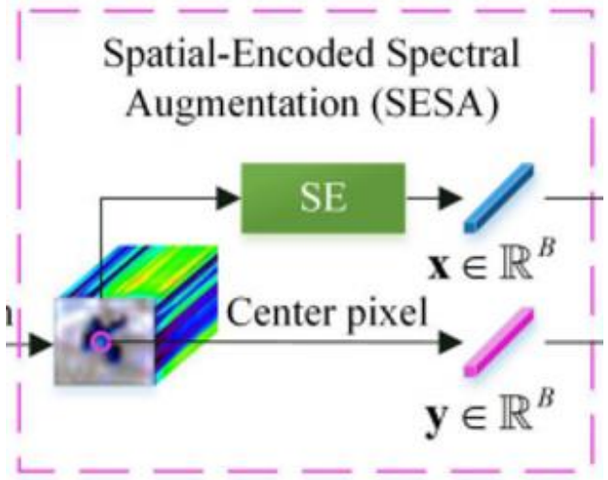
$$c\left(\mathbf{a}, \mathbf{b}\right) = \left(\mathbf{a}^{\mathrm{T}}\mathbf{b}\right) / \left(\|\mathbf{a}\|_2 \cdot \|\mathbf{b}\|_2\right)$$

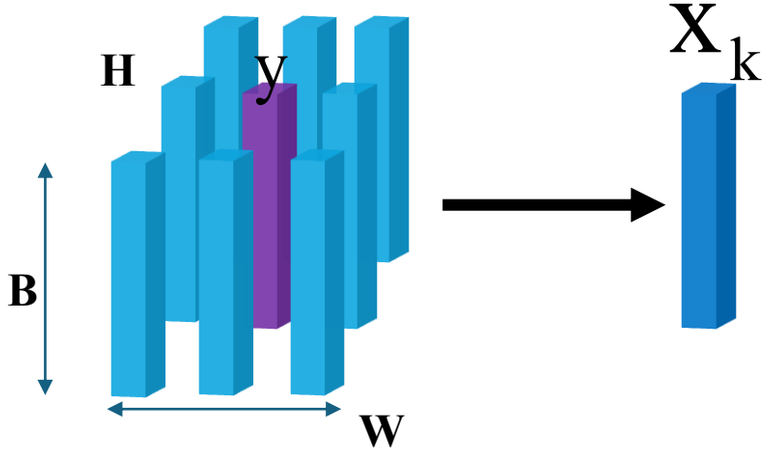
在SESA部分，是通过将中心像素周围的临近像素进行数据增强，来构建正负样本对。这边会存在一个问题：假设你的目标像素就只有2个像素，如果你的patch取了11，那么你的临近像素会有11×11-1共120个像素，而其中有119个像素全是“背景”像素，通过数据增强会引入很大的噪声，影响正样本对构建的有效性。

3.3 可能的模型模块优化

香 港 大 學
THE UNIVERSITY OF HONG KONG

1.SESA → 目标感知加权





$$\mathbf{x_k} = \mathbf{P_k^T W}$$

$$\mathbf{W}_i = \frac{\exp\left(c(\mathbf{y}, \mathbf{P}_i)\right)}{\sum_{j=1}^{p^2} \exp\left(c\left(\mathbf{y}, \mathbf{P}_j\right)\right)}$$

$$c\left(\mathbf{a}, \mathbf{b}\right) = \left(\mathbf{a}^{\mathrm{T}}\mathbf{b}\right) / \left(\|\mathbf{a}\|_2 \cdot \|\mathbf{b}\|_2\right)$$

原 SESA 是纯 cosine softmax，容易被背景像素"搭便车"。可以在 Data_Patch.py的 patch_encoded里加一个空间高斯衰减先验：中心像素权重天然高、越往外越低，再跟 cosine 乘。公式改成 W_i = softmax(cosine(y, P_i) + λ·G(i))，G(i) 是像素 i 到中心的高斯 log 权重，λ 控制空间先验强度。这样即使 patch 里背景占多数，中心区域（真目标可能在的位置）话语权也更高。对 G2A 小目标，λ 设 0.5~1.0 试。

3.3 可能的模型模块优化

2.对比损失升级：Focal-InfoNCE 或小目标难例加权

CL_Train.py的 info_nce_loss现在是对称的交叉熵，没区分"这个负对是不是难例"。小目标场景下，大多数负对是"背景-背景"，很容易分，loss 被这些 easy 样本主导。改成：

$$\ell_i = -\log\left[\exp(c(x_i,y_i)/\alpha) \,/\, \Sigma_j \exp(c(x_i,y_j)/\alpha) \right] * w_i$$

$$w_i = (1 - p_i)^{\gamma},\ p_i = \exp(c(x_i,y_i)/\alpha) \,/\, \Sigma_j \exp(c(x_i,y_j)/\alpha)$$

就是 Focal 的套路，尝试$\gamma=2$ 。让模型在 batch 里更关注"那些跟正对相似度还不低"的难负对——小目标最容易跟背景里某些相似材质（比如天空反光、建筑屋顶）混淆，Focal 能让这些对被重点推